

BoxAdmin — Fase 3: Self-service del alumno (PWA)

Especificación técnica para agente de desarrollo (Claude Code)

Septiembre 2026

Contexto

Con el núcleo operativo (Fase 1) y el motor de recurrencia (Fase 2) funcionando, esta fase habilita al **alumno** a operar sin depender del admin: ver su calendario, reservar/cancelar clases sueltas dentro de las reglas configuradas, subir comprobante de pago, y auto-registrarse con una clave de invitación. Es la primera fase que introduce **frontend** (Next.js como PWA).

Objetivo de esta fase: un alumno entra a `https://<slug-del-salon>.boxadmin.app` (o `/app/<slug>`), se loguea (o se auto-registra), ve su calendario de clases, reserva/cancela dentro de las reglas, y sube un comprobante de pago.

Criterio de éxito: un alumno de prueba completa el ciclo completo self-service sin que el admin intervenga manualmente: auto-registro → elegir pack → elegir rutina inicial → ver sus próximas clases → cancelar una → subir comprobante.

1. Stack e instalación (frontend)

```
cd boxadmin/apps
pnpm dlx create-next-app@latest web --typescript --tailwind --app --src-dir --import-alias
  ↪ "@/*"
cd web
pnpm add @tanstack/react-query axios zod react-hook-form @hookform/resolvers
pnpm add next-pwa
pnpm add -D @types/node
```

`next.config.ts` — habilitar PWA:

```
import withPWA from "next-pwa";

const withPWAConfig = withPWA({
  dest: "public",
  register: true,
  skipWaiting: true,
  disable: process.env.NODE_ENV === "development",
});

export default withPWAConfig({
  reactStrictMode: true,
});
```

Agregar `public/manifest.json` con nombre, ícono e ícono de instalación de “BoxAdmin”.

Storage de comprobantes: usar un bucket S3-compatible (Backblaze B2 o DigitalOcean Spaces, según la decisión de hosting):

```
# backend
```

```
npm add @aws-sdk/client-s3 @aws-sdk/s3-request-presigner
```

2. Modelo de datos de esta fase (Prisma)

```
model ClaveInvitacion {
  id          String   @id @default(cuid())
  tenantId    String
  tenant      Tenant   @relation(fields: [tenantId], references: [id])

  codigo      String   @unique
  activa      Boolean   @default(true)
  usosMax     Int?      // null = ilimitado
  usosActuales Int      @default(0)

  createdAt   DateTime @default(now())

  @@index([tenantId])
  @@map("claves_invitacion")
}

model Comprobante {
  id          String   @id @default(cuid())
  tenantId    String
  perfilId    String
  perfil      Perfil   @relation(fields: [perfilId], references: [id])

  urlArchivo  String
  estado      String   @default("pendiente") // pendiente | aprobado | rechazado
  revisadoPor String?
  revisadoEn  DateTime?
  nota        String?

  createdAt   DateTime @default(now())

  @@index([tenantId, perfilId])
  @@map("comprobantes")
}

model ListaEspera {
  id          String   @id @default(cuid())
  tenantId    String
  turnoId     String
  turno      Turno    @relation(fields: [turnoId], references: [id])
  perfilId    String
  perfil      Perfil   @relation(fields: [perfilId], references: [id])

  posicion    Int
  notificado  Boolean   @default(false)

  createdAt   DateTime @default(now())

  @@unique([turnoId, perfilId])
  @@index([tenantId, turnoId])
  @@map("listas_espera")
}
```

Agregar a Perfil el flag `autoRegistrado Boolean @default(false)` para marcar altas pendientes de revisión del admin, tal como se relevó en TurnoFit.

Migración:

```
pnpm dlx prisma migrate dev --name fase3_self_service
```

3. Endpoints nuevos de esta fase (backend)

3.1 Auto-registro

Método	Ruta
POST	/invitaciones
PATCH	/invitaciones/:id
POST	/auth/auto-registro
GET	/usuarios?autoRegistrado=true

3.2 Self-service de calendario (rol ALUMNO)

Método	Ruta
GET	/mi-calendario?desde=&hasta=
GET	/turnos-disponibles?salaId=&desde=&hasta=
POST	/turnos/:id/mi-reserva
DELETE	/mis-reservas/:id
POST	/turnos/:id/lista-espera
DELETE	/lista-espera/:id

3.3 Comprobantes

Método	Ruta
POST	/comprobantes
GET	/comprobantes?estado=pendiente
PATCH	/comprobantes/:id/aprobar
PATCH	/comprobantes/:id/rechazar

4. Lógica de negocio clave de esta fase

- **Lista de espera vs. “solo cupos liberados”:** mantener ambos mecanismos (así funciona hoy en TurnoFit y cambiarlo rompería expectativas), pero unificarlos bajo un único servicio `Disponibilidad-Service` que calcula, para un turno y un perfil dado, un estado consolidado: `LIBRE` | `SOLO_ADMIN` | `LISTA_ESPERA` | `LLENO`. Esto es la mejora de UX que identificaste: la lógica sigue siendo la misma de TurnoFit, pero se expone al frontend como un solo concepto en vez de dos features separadas y confusas.

- Al liberarse un cupo (cancelación), el ReservasService debe: 1) verificar si hay ListaEspera para ese turno, 2) si notificado=false para el primero en la cola, asignarlo automáticamente o notificarlo, según configuración del tenant (a definir en Fase 5 con el módulo de notificaciones — dejar el hook preparado en esta fase).
- Ventanas mínimas de anotación/cancelación: leer primero de Sala, si es null, heredar de la config general del tenant (a implementar como un modelo ConfiguracionTenant simple si no existe aún — puede resolverse con un JSON en Tenant o una tabla propia; para esta fase alcanza con campos directos en Tenant).

5. Pantallas del frontend (Next.js/PWA)

```
apps/web/src/app/
├── (publico)/
│   ├── [slug]/registro/page.tsx      # auto-registro con clave de invitación
│   └── [slug]/login/page.tsx
└── (alumno)/
    ├── [slug]/
    │   ├── calendario/page.tsx      # vista semanal, reservar/cancelar
    │   ├── mi-pack/page.tsx         # resumen de clases disponibles/usadas
    │   ├── comprobantes/page.tsx    # subir y ver estado de comprobantes
    │   └── perfil/page.tsx
```

Usar React Query para el estado de servidor (calendario, reservas) con invalidación optimista al reservar/cancelar, para que la UI se sienta instantánea — es uno de los puntos donde TurnoFit se siente pesado según lo que describiste.

6. Checklist de aceptación de la Fase 3

- ☐ Un alumno puede auto-registrarse con una clave de invitación válida, eligiendo pack.
- ☐ El alta queda marcada como autoRegistrado=true y aparece en el listado de pendientes del admin.
- ☐ Un alumno logueado ve su calendario y puede reservar/cancelar una clase suelta respetando las ventanas mínimas configuradas.
- ☐ Reservar un turno lleno ofrece anotarse en lista de espera en vez de un error genérico.
- ☐ Cancelar un cupo con lista de espera dispara la asignación (o notificación) del primero en la cola.
- ☐ Un alumno puede subir un comprobante y verlo en estado “pendiente”; el admin puede aprobarlo/rechazarlo.
- ☐ La PWA es instalable (manifest + service worker) y funciona razonablemente offline para la vista de “mi calendario” ya cacheada.
- ☐ Tests e2e: auto-registro completo, reserva propia, cancelación propia, anotación en lista de espera, subida de comprobante.

7. Qué NO entra en esta fase

Notificaciones push/email reales (solo se deja el hook preparado), profesor vinculado al turno, pagos con pasarela, estadísticas — llegan en fases posteriores.